

React Frontend - Part 2 (Bootstrap, Hooks, Lifecycle, Forms & Lists) - Problem Statement

Overview

In this week, you will upgrade your last project's static React app to use **Bootstrap 5** for a professional UI and deepen your understanding of core React concepts: `useState`, `useEffect`, component lifecycle, controlled forms, and rendering lists with keys. By the end of this week, your EasyMart frontend will have polished Bootstrap styling, working search and filter controls, a controlled review form, and well-managed side-effects via `useEffect`.

Learning Objectives

By the end of this week, you will be able to:

- Install and integrate Bootstrap 5 into a Vite/React project
 - Style components using Bootstrap utility classes and components
 - Understand the React component lifecycle and how `useEffect` maps to it
 - Use `useEffect` for side effects: timers, cleanup, and dependency arrays
 - Build fully controlled form inputs (`value` + `onChange`)
 - Validate form inputs with per-field error messages
 - Render lists correctly with unique `key` props
 - Apply conditional rendering patterns in JSX
-

Required Project Structure

Start from your last project (or scaffold fresh) and update the following:

```
├─ src/
│   └─ App.jsx           ← Update: add isLoading state +
useEffect spinner
│   └─ App.css           ← Update: minimal custom styles
(hero gradient, card hover)
│   └─ main.jsx          ← Update: import Bootstrap CSS
│   └─ data/
│       └─ products.js   ← Keep from last project (10
MOCK_PRODUCTS)
│   └─ utils/
│       └─ constants.js  ← Update: add SORT_OPTIONS,
ITEMS_PER_PAGE
```

<pre> └─ helpers.js (formatPrice, formatDate, etc.) └─ components/ └─ layout/ └─ Navbar.jsx └─ Footer.jsx └─ common/ └─ Notification.jsx useEffect auto-dismiss └─ products/ └─ ProductCard.jsx └─ ProductSearch.jsx (controlled) └─ Pagination.jsx keys └─ ProductList.jsx └─ ProductDetail.jsx controlled review form └─ components/auth/ └─ LoginForm.jsx controlled inputs └─ pages/ └─ products/ └─ HomePage.jsx featured grid </pre>	← Keep from last project ← Replace: Bootstrap navbar ← Replace: Bootstrap footer ← Update: Bootstrap alert + ← Replace: Bootstrap card ← Update: Bootstrap form controls ← Update: Bootstrap pagination + ← Update: Bootstrap grid + keys ← Update: Bootstrap layout + ← Update: Bootstrap form + ← Update: Bootstrap hero +
--	---

Setup Instructions

Step 1: Start from last project

Copy your last project as your starting point.

Step 2: Install Bootstrap

```
npm install bootstrap
```

Step 3: Import Bootstrap in `main.jsx`

```
import 'bootstrap/dist/css/bootstrap.min.css';
```

Step 4: Remove custom CSS from last project (optional)

You can remove or significantly reduce the custom CSS from the last project since Bootstrap handles most utility styling. Keep only the custom `.hero-section` gradient and any non-Bootstrap-achievable styles.

Tasks Overview

Part 1: Bootstrap Setup & Layout Components

Task 1.1 — `main.jsx`: Bootstrap import

Update `main.jsx` to import `bootstrap/dist/css/bootstrap.min.css` before `App.css`. This loads all Bootstrap utility classes globally.

Task 1.2 — `App.css`: Clean up custom styles

Remove duplicate CSS that Bootstrap now handles. Keep only:

- `.hero-section` gradient background
- `.main-content` top padding
- `.clickable` cursor pointer
- `.text-star` for gold star color

Task 1.3 — `Navbar.jsx`: Bootstrap Navbar

Replace the custom navbar with a Bootstrap navbar:

- Use `navbar navbar-expand-lg navbar-dark bg-dark sticky-top shadow-sm`
- Wrap navigation links in `navbar-nav flex-row`
- Use Bootstrap `btn btn-outline-light btn-sm` for the login button
- Current page highlight using Bootstrap `active` class

Task 1.4 — `Footer.jsx`: Bootstrap Footer

Create a Bootstrap footer:

- Use `bg-dark text-light py-4` container
 - Three-column layout with `row g-4`: brand column, categories column, contact column
 - Copyright row at the bottom with `border-top border-secondary`
-

Part 2: React Hooks Deep Dive — `useState` & `useEffect`

Task 2.1 — `App.jsx`: `useEffect` Loading Simulation

Add an `isLoading` state that starts `true` and is set to `false` after 800ms using `useEffect`. This demonstrates:

- `useEffect` with an empty dependency array (`[]`) — runs once on mount (like `componentDidMount`)
- `return () => clearTimeout(id)` — cleanup function to prevent memory leaks (like `componentWillUnmount`)
- While loading, render a Bootstrap `spinner-border`

```
const [isLoading, setIsLoading] = useState(true);

useEffect(() => {
  const id = setTimeout(() => setIsLoading(false), 800);
  return () => clearTimeout(id); // cleanup
}, []); // empty array = run once on mount
```

Task 2.2 — Notification.jsx: `useEffect` Auto-dismiss

Update the `Notification` component so it auto-dismisses after 3 seconds:

- Use `useEffect` that depends on `[message]` — reruns when a new notification arrives
- Set a 3-second timeout to clear the notification
- Return the cleanup function `() => clearTimeout(id)` to cancel pending timers
- Render as a Bootstrap `alert alert-success alert-dismissible` (or `alert-danger` for errors)

```
useEffect(() => {
  if (!message) return;
  const id = setTimeout(() => onDismiss(), 3000);
  return () => clearTimeout(id); // cancel if message changes before 3s
}, [message]);
```

Task 2.3 — App.jsx: Hash-based Navigation (keep from last project)

Keep the same hash navigation as the last project (`popstate` listener, `history.pushState`). This is intentional — it reinforces that routing is a concern separate from styling. React Router will be introduced in the next week.

Part 3: Controlled Forms

Task 3.1 — LoginForm.jsx: Controlled Login Form

Convert the login and register forms to fully controlled components:

- Each field: `value={form.email} + onChange={(e) => setForm(p => ({...p, email: e.target.value}))}`
- Validate on submit — show per-field error messages using Bootstrap's `is-invalid` class and `invalid-feedback` div
- Show/hide password toggle using a `useState` boolean

```
// Controlled input pattern
<input
  type="email"
  className={`form-control ${errors.email ? 'is-invalid' : ''}`}
  value={form.email}
  onChange={(e) => setForm(prev => ({ ...prev, email: e.target.value })))}
/>
{errors.email && <div className="invalid-feedback">{errors.email}</div>}
```

Use Bootstrap **Nav Tabs** (`nav nav-tabs`) to switch between Login and Register panels without navigating away.

Task 3.2 — ProductSearch.jsx: Controlled Search & Filters

All search and filter inputs must be controlled:

- Search: `<input value={searchTerm} onChange={...} />`
- Category dropdown: `<select value={selectedCategory} onChange={...}>` with `<option>` elements using `key={category}`
- Sort: `<select value={sortBy} onChange={...}>` — iterate `SORT_OPTIONS` with `key` on each option
- Price range: two controlled number inputs
- In-stock checkbox: `<input type="checkbox" checked={inStockOnly} onChange={(e) => setInStockOnly(e.target.checked)} />`

Demonstrate **Lists and Keys** on the dropdown options:

```
{SORT_OPTIONS.map(({ value, label }) => (
  <option key={value} value={value}>{label}</option> // key = unique value
))}
```

Task 3.3 — ProductDetail.jsx: Controlled Review Form

Add a review submission form to the product detail view:

- Rating: rendered as radio inputs or star buttons, controlled with `useState`
- Review title and body: controlled `<input>` and `<textarea>`
- Validate: title required, body minimum 10 characters, rating required

- On submit: add the review to a local `reviews` state array (no API yet) and reset the form
 - Display existing reviews as a list with `key={review.id}`
-

Part 4: Lists and Keys

Task 4.1 — ProductList.jsx: Bootstrap Grid with Keys

Render the product list with proper keys:

```
{products.map((product) => (  
  <div key={product._id} className="col">  
    <ProductCard product={product} onSelect={onSelectProduct} />  
  </div>  
))}
```

Use Bootstrap responsive grid: `row-cols-1 row-cols-sm-2 row-cols-md-3 row-cols-lg-4 g-4`

Show an empty state when no products match filters:

```
{products.length === 0 && (  
  <div className="col-12 text-center py-5">  
    <p className="text-muted fs-5">No products found. Try adjusting your  
filters.</p>  
  </div>  
)}
```

Task 4.2 — Pagination.jsx: Bootstrap Pagination with Keys

Build Bootstrap pagination:

```
<ul className="pagination">  
  {pages.map((page) => (  
    <li key={page} className={`page-item ${page === currentPage ? 'active' : ''}`}>  
      <button className="page-link" onClick={() => onPageChange(page)}>  
{page}</button>  
    </li>  
  ))}  
</ul>
```

Task 4.3 — ProductCard.jsx: Bootstrap Card

Style each product card with Bootstrap:

- `card h-100 shadow-sm` with hover effect via CSS
- `card-img-top` for the product image
- `card-body d-flex flex-column` so the button stays at the bottom
- Badge for stock status: `badge bg-success` / `badge bg-danger`
- Use `formatPrice` from helpers for price display

Part 5: Bootstrap Components

Task 5.1 — HomePage.jsx: Hero Section + Featured Grid

Build the homepage with:

- A large hero `<div>` with a gradient background and centered Bootstrap typography
- A "Shop Now" button using `btn btn-light btn-lg`
- A featured products section rendering 4 products using `row-cols-1 row-cols-md-4 g-4`
- Four feature highlight cards (Free Shipping, Secure Payments, etc.) using Bootstrap `card text-center`

Task 5.2 — Modals & Badges (Bonus)

Add a Bootstrap `modal` for the product quick-view — trigger it with a "Quick View" button on `ProductCard`. Display product image, name, price, and an "Add to Cart" button inside the modal.

What You Can Verify in the Browser

Run `npm run dev` and open `http://localhost:3000`. Open DevTools (F12) and keep the **Console** tab visible.

What's new in this week:

- Bootstrap 5 responsive UI replaces hand-written CSS
- Collapsible navbar on mobile (hamburger menu)
- Loading spinner on first render (`useEffect` + `setTimeout`)
- Form validation with inline red error messages (`is-invalid`)
- `useEffect` cleanup pattern demonstrated in `Notification`
- Console shows **zero** key-prop warnings on lists

Scope is the same as the last week: No real cart page, no real auth, hash navigation.

Home Page

Action	Expected result
--------	-----------------

Action	Expected result
Open <code>http://localhost:3000</code>	Bootstrap spinner loads for ~0.8 s, then home page appears
Home page layout	Gradient hero banner, Shop Now button, 4 featured product cards, 4 feature highlight cards (Free Shipping, etc.)
Click Shop Now	Navigates to Products page
Resize window below 992 px	Navbar links collapse into hamburger ☰ icon
Click ☰	Navbar menu toggles open/closed (Bootstrap JS)

Products & Filtering

Action	Expected result
Click Products in navbar	Bootstrap card grid (up to 8 per page)
Type in search box	Cards filter in real time
Select a category	Only that category shown
Sort "Price: Low to High"	Cheapest card first
Set min/max price	Out-of-range products hidden
Tick In Stock Only	Out-of-stock cards hidden
All filters active → Clear Filters	All reset, full list returns
No results	"No products found. Try adjusting your filters." message (not a blank grid)

Product Detail

Action	Expected result
Click any product card	Full Bootstrap layout: image left, info right on desktop; stacked on mobile
– at qty 1	Button greyed out; quantity stays at 1
+ at max stock	Quantity stops at stock value
Add to Cart (not logged in)	Error toast: "Please log in to add items to your cart."
Add to Cart (logged in)	Success toast: "{Product} (xN) added to cart!"

Action	Expected result
Write a Review → submit empty	Red is-invalid border + "Comment is required" message
Submit valid review	Review appears in list with star rating and date

Authentication (Controlled Form)

Action	Expected result
Click Login	Bootstrap-styled Login / Register tab form
Click Register tab	Register fields appear (no page reload)
Submit Login with empty fields	Red borders + error messages under each field
Submit Login with invalid email	Email-specific error message
Submit Login with valid data	Logged in; "Hello, {name}" + Logout in navbar
Submit Register with mismatched passwords	"Passwords do not match" error
Click Logout	Auth cleared; navbar reverts

Notifications

Action	Expected result
Trigger any notification	Bootstrap alert banner appears at top
Wait 3 seconds	Alert auto-dismisses (useEffect cleanup timer)
Click ✕	Alert dismisses immediately

Lists & Keys (Console Check)

Action	Expected result
Open Console (F12) while browsing	No "Each child in a list should have a unique key" warning
Pagination	Page 1 = first 8 products; Prev disabled; clicking 2 or Next shows the rest
Last page	Next button disabled